Univerzitet u Zenici
Politehnički fakultet

BILJEŠKE S PREDAVANJA · SEDMICA 3

Upravljanje konfiguracijom i kontejneri kao kod

Ansible (inventory, playbook, role, šabloni, Vault), povezivanje Terraforma i Ansible-a te kontejneri kao kod: `Dockerfile`, `Docker Compose` i uvod u Kubernetes manifeste.

PREDMET	Računarstvo u oblaku
STUDIJ	II ciklus studija, III semestar, akademska godina 2026/27
NASTAVNIK	V. prof. dr. Đulaga Hadžić
ASISTENT	ass. Mr. Adin Jahić
TERMIN	petak, 30.10.2026, 17:15–20:15, Kabinet 3204
TEMA	Upravljanje konfiguracijom i kontejneri kao kod

OKVIR NASTAVNE JEDINICE

Ciljevi i način rada

U drugoj sedmici su mašine napravljene iz koda: Terraform ih je kreirao, cloud-init personalizovao, a Packer pripremio sliku. Ostalo je pitanje šta se događa *unutar* mašine nakon toga: paketi, konfiguracijske datoteke, servisi, tajne. Taj dio rješava **upravljanje konfiguracijom**, a ova sedmica ga predstavlja kroz Ansible. Druga polovina sedmice prenosi isti način razmišljanja na kontejnere: aplikacija i njeno okruženje se opisuju datotekama (`Dockerfile`, `compose.yaml`, Kubernetes manifesti), a ne naredbama upisanim ručno.

Nakon nastavne jedinice student je u stanju:

- objasniti razliku između provisioninga (Terraform) i upravljanja konfiguracijom (Ansible) i povezati ih u jedan tok,
- napisati inventory, playbook i šablon (Jinja2) te provjeriti idempotentnost izvršavanja,
- organizovati kod u rolu i zaštititi osjetljive vrijednosti pomoću Ansible Vaulta,
- generisati Ansible inventory iz Terraform izlaza,
- napisati Dockerfile i Compose datoteku za aplikaciju s bazom i objasniti vezu slojeva slike s prvom sedmicom,
- pročitati i napisati osnovni Kubernetes manifest (Deployment, Service, ConfigMap) i objasniti razliku između Terraform i Kubernetes pristupa usklađivanju stanja.

Preduslovi za izvođenje isječaka

- Linux ili WSL2 (Ubuntu 24.04) s pravom `sudo`, Docker (Docker Desktop s WSL integracijom ili Docker Engine) i rezultat druge sedmice (`~/iac/s2/p3`) za primjer 3.

- Primjeri 1 i 2 se izvršavaju na lokalnoj mašini (`ansible_connection=local`), pa ne traže VM. Za nginx kao servis u WSL2 treba uključen `systemd` (`/etc/wsl.conf`: `[boot] systemd=true`, zatim `wsl --shutdown`).
- Svaki primjer ima vlastiti direktorij pod `~/iac/s3/`.

bash — Isječak 0 · priprema

```
sudo apt update && sudo apt install -y ansible curl
ansible --version
docker version --format '{{.Server.Version}}' && docker compose version

# kubectl i kind (samo za posljednji dio)
curl -Lo kubectl "https://dl.k8s.io/release/$(curl -Ls
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install kubectl /usr/local/bin/kubectl
curl -Lo kind https://kind.sigs.k8s.io/dl/v0.24.0/kind-linux-amd64 && sudo install kind
/usr/local/bin/kind

mkdir -p ~/iac/s3 && cd ~/iac/s3
```

---

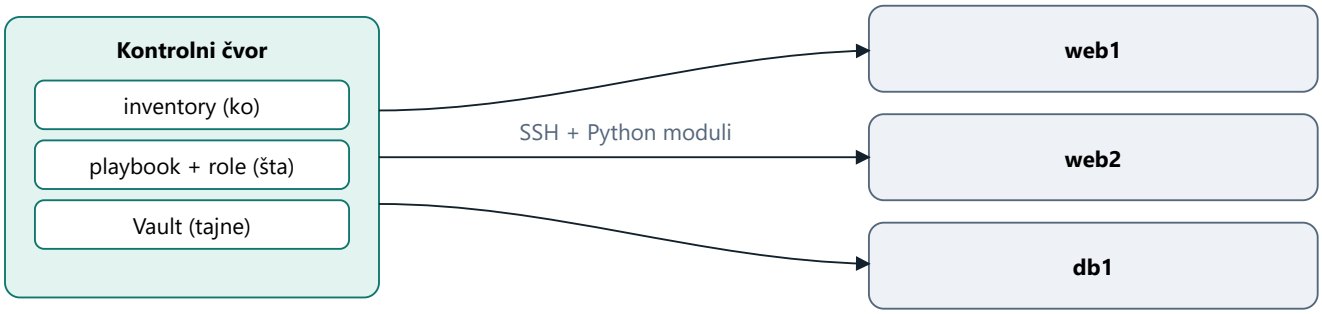
1 · POJAM

Od provisioninga do upravljanja konfiguracijom

Terraform odgovara na pitanje *koji resursi postoje* (mašine, mreže, diskovi). Ne bavi se onim što je unutar mašine. cloud-init to rješava samo pri prvom pokretanju, a promjene u radu (nova verzija konfiguracije, novi korisnik, zakrpa) ostaju bez vlasnika. Za taj dio služi **alat za upravljanje konfiguracijom**.

Ansible je takav alat sa tri osobine koje ga čine pogodnim za početak:

- **Bez agenta.** Upravljana mašina ne treba nikakav poseban softver. Ansible se spaja preko SSH-a i izvršava module (Python) na mašini.
- **Push model.** Kontrolni čvor (laptop ili CI) šalje konfiguraciju na mašine kad se izda naredba.
- **Deklarativni moduli.** Zadatak ne kaže "izvrši `apt install nginx`", nego "paket `nginx` treba biti prisutan". Modul sam provjeri stanje i mijenja ga samo ako treba. To je idempotentnost iz druge sedmice.



Slika 1. Ansible nema agenta: kontrolni čvor se preko SSH-a spaja na svaku mašinu iz inventorya i izvršava zadatke iz playbooka.

POJAM	ZNAČENJE
Inventory	Spisak mašina podijeljenih u grupe (`[web]`, `[db]`) uz pripadajuće varijable.
Modul	Jedinica posla (`apt`, `service`, `template`, `user`, `file`, `copy`...). Većina je idempotentna.
Task	Jedan poziv modula s parametrima.
Play / playbook	Play veže grupu mašina (`hosts`) s listom zadataka. Playbook je YAML datoteka s jednim ili više playeva.
Handler	Zadatak koji se izvršava samo ako ga je neki zadatak "obavijestio" (`notify`) da je nešto promijenio, npr. reload servisa nakon izmjene konfiguracije.
Role	Pakovanje zadataka, šablona, varijabli i handlera u višekratno upotrebljivu cjelinu.
Fact	Podatak koji Ansible prikupi o mašini (OS, adrese, memorija). Koristi se u uslovima i šablonima.
Vault	Šifrovanje osjetljivih datoteka i vrijednosti u repozitoriju.

GRANICA IDEMPOTENTNOSTI

Moduli `command` i `shell` ne znaju ništa o stanju, pa su uvijek `changed`. Koriste se tek kad ne postoji odgovarajući modul, uz `creates:` ili `changed_when:` koji im vraćaju provjeru stanja.

2 · PRIMJER 1

Inventory, playbook i šablon

Prvi primjer postavlja nginx s vlastitom početnom stranicom. Cilj je lokalna mašina (`ansible_connection=local`), što znači da isti kod radi na laptopu, u WSL2 ili na VM iz druge sedmice; mijenja se samo inventory.

bash + yaml — Isječak 1 · inventory, šablon, playbook

```
mkdir -p ~/iac/s3/p1/templates && cd ~/iac/s3/p1

cat > inventory.ini <<'EOF'
[web]
localhost ansible_connection=local
EOF

cat > templates/index.html.j2 <<'EOF'
<h1>{{ naslov }}</h1>
<p>Host: {{ ansible_facts['hostname'] }},
   OS: {{ ansible_facts['distribution'] }} {{ ansible_facts['distribution_version'] }}</p>
<p>Datoteku generiše Ansible. Ručne izmjene će biti prebrisane.</p>
EOF

cat > site.yml <<'EOF'
- name: Web server
  hosts: web
  become: true
  vars:
    naslov: "Računarstvo u oblaku, sedmica 3"
  tasks:
    - name: nginx je instaliran
      ansible.builtin.apt:
        name: nginx
```

```
        state: present
        update_cache: true
        cache_valid_time: 3600

    - name: početna stranica iz šablona
      ansible.builtin.template:
        src: index.html.j2
        dest: /var/www/html/index.html
        mode: "0644"
      notify: reload nginx

    - name: nginx radi i uključen je pri pokretanju
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true

  handlers:
    - name: reload nginx
      ansible.builtin.service:
        name: nginx
        state: reloaded
EOF
```

Prije stvarnog izvršavanja radi se provjera, a zatim se playbook pokreće dva puta (parametar `-K` traži `sudo` lozinku za `become`):

bash — Isječak 2 · pokretanje i idempotentnost

```
ansible -i inventory.ini web -m ping                    # veza i Python na mašini
ansible-playbook -i inventory.ini site.yml --syntax-check
ansible-playbook -i inventory.ini site.yml -K --check --diff   # "suho" izvršavanje: šta bi se
promijenilo
ansible-playbook -i inventory.ini site.yml -K           # 1. izvršavanje
ansible-playbook -i inventory.ini site.yml -K           # 2. izvršavanje
curl -s http://localhost
```

izlaz (skraćeno, brojevi se mogu razlikovati)

```
PLAY RECAP *********************************************************
localhost : ok=5  changed=3  unreachable=0  failed=0     # 1. izvršavanje
...
PLAY RECAP *********************************************************
localhost : ok=4  changed=0  unreachable=0  failed=0     # 2. izvršavanje
```

Najvažniji red je `changed=0` u drugom izvršavanju: playbook je idempotentan. Isto tako se može provjeriti i *drift* konfiguracije: ručno izmijeniti `/var/www/html/index.html` i ponovo pokrenuti playbook. Zadatak `template` će prijaviti `changed`, vratiti sadržaj i pozvati handler (`reload nginx`). Handler se izvršava *samo* kad je prethodni zadatak nešto promijenio, što je i razlog zašto se reload ne radi pri svakom pokretanju.

Inventory, grupe i varijable

Isti inventory se može napisati u YAML-u. Varijable se ne drže u njemu nego u direktorijima `group_vars/<grupa>/` i `host_vars/<host>/` pored inventorya, pa se podaci i spisak mašina održavaju odvojeno.

yaml — Isječak 3 · inventory.yml (ilustracija)

```yaml
all:
  children:
    web:
      hosts:
        web1: { ansible_host: 192.168.122.57, ansible_user: student }
        web2: { ansible_host: 192.168.122.58, ansible_user: student }
    db:
      hosts:
        db1:  { ansible_host: 192.168.122.60, ansible_user: student }
# pregled: ansible-inventory -i inventory.yml --graph
# varijable grupe web: group_vars/web/vars.yml
```

Najčešći moduli i konstrukti

POTREBA	PRIMJER
Paket	`ansible.builtin.apt: { name: [nginx, curl], state: present }`
Servis	`ansible.builtin.service: { name: nginx, state: restarted }`
Datoteka iz šablona / kopija	`template` (Jinja2), `copy` (statična)
Korisnik, direktorij	`user`, `file` (`state: directory`, `mode`, `owner`)
Jedna linija u datoteci	`lineinfile`
Uslov, petlja	`when: ansible_facts['os_family'] == 'Debian'`, `loop: [a, b]`
Jinja2	`{{ var }}`, `{% for h in groups['web'] %}`, filteri: `{{ x | default('y') }}`

3 · PRIMJER 2

Role i Ansible Vault

Kad playbook naraste, zadaci i datoteke se pakuju u **rolu**. Rola ima fiksnu strukturu direktorija, pa je lako ponovo upotrijebiti i dijeliti. Drugi primjer iz prethodnog pravi rolu `webserver` i dodaje *tajnu* (lozinku baze) koja se šifruje Vaultom, tako da u Gitu nikad ne završi u čistom tekstu.

DIREKTORIJ ROLE	SADRŽAJ
`tasks/main.yml`	Zadaci role.
`handlers/main.yml`	Handleri.
`templates/`, `files/`	Šabloni (Jinja2) i statične datoteke.
`defaults/main.yml`	Podrazumijevane vrijednosti varijabli (najniži prioritet, lako se nadjačaju).
`vars/main.yml`	Varijable role višeg prioriteta (rijetko se mijenjaju).

bash + yaml — Isječak 4 · rola webserver

```
mkdir -p ~/iac/s3/p2 && cd ~/iac/s3/p2
mkdir -p roles/webserver/{tasks,handlers,templates,defaults} group_vars/web

cat > ansible.cfg <<'EOF'
[defaults]
inventory = inventory.ini
roles_path = roles
vault_password_file = .vault_pass
EOF
cp ../p1/inventory.ini .

cat > roles/webserver/defaults/main.yml <<'EOF'
naslov: "web server"
app_user: "nginx-app"
EOF

cat > roles/webserver/tasks/main.yml <<'EOF'
- name: nginx je instaliran
  ansible.builtin.apt: { name: nginx, state: present, cache_valid_time: 3600 }

- name: početna stranica
  ansible.builtin.template:
    src: index.html.j2
    dest: /var/www/html/index.html
    mode: "0644"
  notify: reload nginx

- name: direktorij za konfiguraciju aplikacije
  ansible.builtin.file: { path: /etc/ruo, state: directory, mode: "0750" }

- name: tajna aplikacije (samo root može čitati)
  ansible.builtin.template:
    src: app.env.j2
    dest: /etc/ruo/app.env
    owner: root
```

```
    mode: "0600"
  no_log: true                # vrijednost se ne ispisuje u izlazu

- name: nginx radi
  ansible.builtin.service: { name: nginx, state: started, enabled: true }
EOF

cat > roles/webserver/handlers/main.yml <<'EOF'
- name: reload nginx
  ansible.builtin.service: { name: nginx, state: reloaded }
EOF

cat > roles/webserver/templates/index.html.j2 <<'EOF'
<h1>{{ naslov }}</h1><p>{{ ansible_facts['hostname'] }}</p>
EOF

cat > roles/webserver/templates/app.env.j2 <<'EOF'
DB_USER=appuser
DB_PASSWORD={{ db_password }}
EOF

cat > site.yml <<'EOF'
- hosts: web
  become: true
  roles:
    - webserver
EOF
```

Vault

Prema dogovoru, obična varijabla u `vars.yml` samo *upućuje* na šifrovanu varijablu s prefiksom `vault_`. Tako se u kodu odmah vidi odakle vrijednost dolazi, a pretraga (`grep`) ostaje moguća.

bash — Isječak 5 · šifrovanje tajne

```
echo "db_password: \"{{ vault_db_password }}\"" > group_vars/web/vars.yml
echo "vault_db_password: \"LabLozinka-123\"" > group_vars/web/vault.yml

echo "lozinka-za-vault-lab" > .vault_pass && chmod 600 .vault_pass
printf '.vault_pass\n' > .gitignore                  # lozinka za Vault NIKAD u Git

ansible-vault encrypt group_vars/web/vault.yml        # koristi vault_password_file iz ansible.cfg
head -c 60 group_vars/web/vault.yml; echo
ansible-vault view group_vars/web/vault.yml
ansible-playbook site.yml -K
```

izlaz (skraćeno)

```
$ANSIBLE_VAULT;1.1;AES256
3361383761663839...
-- view:
vault_db_password: "LabLozinka-123"
-- playbook: zadatak "tajna aplikacije" ispisuje: "censored: no_log"
```

Datoteka u repozitoriju je sada šifrovana (AES256) i može se commitovati. Dešifrovanje radi samo ko ima Vault lozinku, a ona se čuva izvan repozitorija (u CI sistemu kao tajna, u upravitelju lozinki ili u datoteci s pravom `0600` koja je u `.gitignore`). Za pojedinačnu vrijednost umjesto cijele datoteke služi `ansible-vault encrypt_string`

`'vrijednost' --name 'vault_api_key'`. Za veće timove se obično uvodi i poseban upravitelj tajni (npr. HashiCorp Vault), a Ansible Vault ostaje u malim projektima.

> **PRAVILO**
>
> Tajna se ne smije pojaviti u *dnevniku* (`no_log: true`), u *Gitu* (Vault ili vanjski upravitelj) ni u *slici* (vidi Dockerfile u 6. poglavlju).

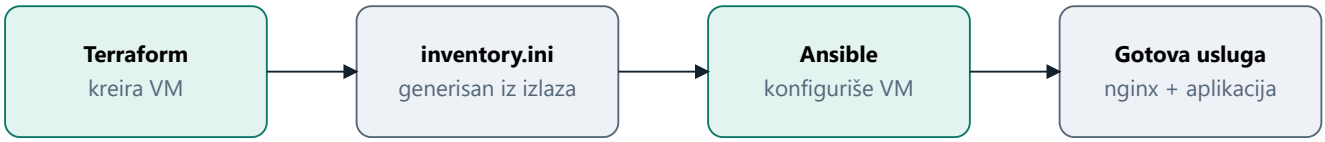
---

4 · TERRAFORM + ANSIBLE

Dva alata, jedan tok

Zajedno čine dvije faze iste automatizacije. Terraform stvara mašine i na izlazu daje njihove adrese. Ansible čita te adrese i konfiguriše mašine. Kritična tačka je prenos podataka: Ansible mora znati *koje* mašine su napravljene.

PRISTUP	KAKO	OCJENA
Terraform generiše inventory	Resurs `local_file` s `templatefile` upisuje `inventory.ini` iz izlaza VM-a. Ansible se pokreće zasebno.	Jednostavno i jasno. **Preporučeno za ovaj predmet.**
`local-exec` provisioner	Terraform sam poziva `ansible-playbook` nakon kreiranja.	Dokumentacija ga svrstava u "krajnju mjeru": greška u Ansible-u ruši `apply`, nije vidljiv u planu.
Dinamički inventory	Ansible plugin (npr. za oblak) sam upita API.	Najelegantnije u oblaku, više podešavanja.



Slika 2. Terraform i Ansible dijele posao po granici "resurs" / "unutrašnjost resursa". Spona među njima je inventory.

---

5 · PRIMJER 3

Terraform stvara VM, Ansible je konfiguriše

Polazi se od primjera 3 iz druge sedmice. cloud-init se svodi na *bootstrap* (korisnik i SSH ključ, dovoljno da se Ansible prijavi), a sve ostalo preuzima rola iz primjera 2.

bash + hcl — Isječak 6 · Terraform generiše inventory

```
mkdir -p ~/iac/s3/p3 && cd ~/iac/s3/p3
cp ../../s2/p3/main.tf .
cp -r ../p2/roles ../p2/group_vars ../p2/ansible.cfg ../p2/.vault_pass .

# cloud-init samo za bootstrap
cat > cloud-init.tftpl <<'EOF'
#cloud-config
hostname: ${hostname}
```

```
users:
  - name: student
    sudo: ALL=(ALL) NOPASSWD:ALL
    shell: /bin/bash
    lock_passwd: true
    ssh_authorized_keys:
      - ${ssh_key}
EOF

# dodatak u main.tf: inventory kao resurs
cat >> main.tf <<'EOF'

resource "local_file" "inventory" {
  filename = "${path.module}/inventory.ini"
  content  = <<-EOT
    [web]
    ${var.vm_name} ansible_host=${libvirt_domain.vm.network_interface[0].addresses[0]}
ansible_user=student
  EOT
}
EOF

cat > site.yml <<'EOF'
- hosts: web
  become: true
  gather_facts: false
  pre_tasks:
    - name: čekaj da SSH postane dostupan
      ansible.builtin.wait_for_connection: { timeout: 300 }
    - name: čekaj da cloud-init završi
      ansible.builtin.command: cloud-init status --wait
      changed_when: false
    - name: prikupi činjenice
      ansible.builtin.setup:
  roles:
    - webserver
EOF

cat > deploy.sh <<'EOF'
#!/bin/bash
set -e
terraform apply -auto-approve                       # 1. resursi + inventory.ini
ANSIBLE_HOST_KEY_CHECKING=False ansible-playbook site.yml   # 2. konfiguracija
EOF
chmod +x deploy.sh && terraform init && ./deploy.sh
curl -s http://$(terraform output -raw ip)
```

izlaz (skraćeno, vrijednosti se razlikuju)

```
Apply complete! Resources: 5 added, 0 changed, 0 destroyed.
...
TASK [webserver : nginx je instaliran] ********************************
changed: [web1]
...
web1 : ok=9  changed=4  unreachable=0  failed=0
<h1>web server</h1><p>web1</p>
```

Dvije stvari koje treba posebno uočiti. **Prva**: inventory je običan resurs u Terraformu, pa se mijenja zajedno s mašinom. Promjenom `vm_name` mijenja se i inventory. **Druga**: Ansible čeka (`wait_for_connection`, `cloud-init`

`status --wait`) jer mašina dobije adresu prije nego je cloud-init završio; bez toga se prvo izvršavanje nasumično završava greškom o nedostupnom SSH-u.

Za usporedbu, varijanta s `local-exec` izgleda ovako, ali joj je mjesto samo u kratkim demonstracijama:

hcl — Isječak 7 · local-exec (ilustracija)

```hcl
resource "terraform_data" "ansible" {
  triggers_replace = [libvirt_domain.vm.id, filemd5("${path.module}/site.yml")]
  depends_on       = [local_file.inventory]

  provisioner "local-exec" {
    command = "ANSIBLE_HOST_KEY_CHECKING=False ansible-playbook site.yml"
  }
}
# problem: greška u Ansible-u ruši apply, a plan ne pokazuje šta će Ansible raditi
```

KO JE ODGOVORAN ZA ŠTA

Ako se promijeni broj mašina ili njihova veličina, mijenja se **Terraform**. Ako se promijeni konfiguracija unutar mašine (paket, datoteka, korisnik), mijenja se **Ansible** i izvršava se samo on. Ta podjela omogućava brze izmjene bez ponovnog kreiranja mašina.

6 · KONTEJNERI KAO KOD

Dockerfile i slojevi

Ista ideja iz prethodnih poglavlja vrijedi za kontejnere: opis okruženja je tekstualna datoteka. **Dockerfile** je recept za izgradnju OCI slike. Svaka instrukcija pravi novi *sloj* (to su slojevi iz prve sedmice, spojeni overlayfs-om), a Docker slojeve **kešira**: ako se instrukcija i sve prije nje nisu promijenile, koristi se postojeći sloj.

INSTRUKCIJA	ZNAČENJE
`FROM`	Osnovna slika. Navodi se konkretna verzija (`python:3.12-slim`), ne `latest`.
`WORKDIR`, `COPY`	Radni direktorij i kopiranje datoteka u sliku.
`RUN`	Naredba pri izgradnji (instalacija zavisnosti). Stvara sloj.
`ENV`, `EXPOSE`	Varijable okruženja i dokumentovanje porta (ne objavljuje ga).
`USER`	Korisnik pod kojim radi proces (ne `root`).
`HEALTHCHECK`	Kako Docker provjerava da li je proces zdrav.
`CMD` / `ENTRYPOINT`	Šta se pokreće. `ENTRYPOINT` je fiksni program, a `CMD` njegovi podrazumijevani argumenti.

Dobra praksa se svodi na nekoliko pravila: stalno promjenjivo ide pri dnu (da keš ostane važeći), `.dockerignore` izbacuje suvišne datoteke, proces ne radi kao root, a tajne se **nikad** ne upisuju u sliku jer ostaju u slojevima čak i kad se kasnije obrišu. *Multi-stage* izgradnja koristi jednu sliku za izgradnju, a drugu, manju, za izvršavanje.

7 · PRIMJER 4

Aplikacija s bazom u Docker Composeu

Jednostavna aplikacija (Flask) broji posjete i čuva brojač u Redisu. Dockerfile opisuje aplikaciju, a **Docker Compose** (`compose.yaml`) deklarativno opisuje cijeli sistem: koje usluge postoje, kako su povezane, koje podatke čuvaju i kada su spremne.

bash + python + dockerfile — Isječak 8 · aplikacija i Dockerfile

```
mkdir -p ~/iac/s3/p4/app && cd ~/iac/s3/p4

cat > app/app.py <<'EOF'
import os
import redis
from flask import Flask

app = Flask(__name__)
r = redis.Redis(host=os.environ.get("REDIS_HOST", "redis"), port=6379)

@app.get("/")
def index():
    return f"Posjeta broj {r.incr('posjete')}\n"

@app.get("/health")
def health():
```

```
    return "ok\n"
EOF

printf 'flask==3.0.3\nredis==5.0.8\ngunicorn==22.0.0\n' > app/requirements.txt
printf '__pycache__/\n*.pyc\n.git/\n' > app/.dockerignore

cat > app/Dockerfile <<'EOF'
# --- faza 1: instalacija zavisnosti (zasebna zbog keša slojeva) ---
FROM python:3.12-slim AS build
WORKDIR /w
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# --- faza 2: finalna slika ---
FROM python:3.12-slim
RUN useradd --system --uid 10001 app
WORKDIR /app
COPY --from=build /install /usr/local
COPY app.py .
USER app
EXPOSE 8000
HEALTHCHECK --interval=10s --timeout=3s --retries=3 \
  CMD python -c "import urllib.request as u; u.urlopen('http://127.0.0.1:8000/health', timeout=2)"
CMD ["gunicorn", "-b", "0.0.0.0:8000", "app:app"]
EOF
```

Zavisnosti se kopiraju i instaliraju *prije* koda aplikacije: dok se `requirements.txt` ne promijeni, svaka izmjena u `app.py` koristi keširane slojeve i izgradnja traje sekundu. Multi-stage ovdje služi uglavnom kao ilustracija; velika korist se vidi kod kompajliranih jezika (Go, Java, C), gdje se kompajler ostavlja u prvoj fazi, a u finalnoj slici ostaje samo binarna datoteka.

yaml — Isječak 9 · compose.yaml

```
cat > compose.yaml <<'EOF'
services:
  web:
    build: ./app
    image: ruo/brojac:1.0
    ports:
      - "8080:8000"
    environment:
      REDIS_HOST: redis
    depends_on:
      redis:
        condition: service_healthy      # web čeka dok Redis ne postane zdrav
    restart: unless-stopped
    networks: [backend]

  redis:
    image: redis:7-alpine
    command: ["redis-server", "--appendonly", "yes"]
    volumes:
      - redisdata:/data                 # podaci prežive brisanje kontejnera
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5
```

```
    networks: [backend]                # port NIJE objavljen: dostupan samo web usluzi

volumes:
  redisdata:

networks:
  backend:
EOF

docker compose config -q && echo "compose.yaml je ispravan"
docker compose up -d --build
docker compose ps
for i in 1 2 3; do curl -s localhost:8080; done
```

izlaz (skraćeno)

```
NAME            SERVICE   STATUS                    PORTS
p4-redis-1      redis     Up 12 seconds (healthy)   6379/tcp
p4-web-1        web       Up 6 seconds (healthy)    0.0.0.0:8080->8000/tcp
Posjeta broj 1
Posjeta broj 2
Posjeta broj 3
```

bash — Isječak 10 · stanje, idempotentnost, čišćenje

```
docker compose up -d                # ništa nije promijenjeno: ništa se ne obnavlja (idempotentno)
docker compose down                 # uklanja kontejnere i mrežu, volumen ostaje
docker compose up -d && curl -s localhost:8080          # "Posjeta broj 4": podaci su preživjeli
ps aux | grep [g]unicorn            # kontejner je običan proces na domaćinu (sedmica 1)
docker compose down -v              # briše i volumen
docker image history ruo/brojac:1.0   # slojevi slike iz Dockerfilea
```

Compose se ponaša isto kao Terraform: poredi željeno i stvarno stanje, a pri ponovnom pokretanju obnavlja samo usluge čija se definicija promijenila. Posebno pogledati tri svojstva datoteke: `healthcheck` + `depends_on` (redoslijed zasnovan na zdravlju, ne samo na pokretanju), mrežu `backend` (Redis nije dostupan izvana) i imenovani volumen (podaci su odvojeni od životnog ciklusa kontejnera).

8 · KUBERNETES

Uvod u manifeste

Compose upravlja kontejnerima na *jednom* računaru. **Kubernetes** upravlja njima na *skupu* računara (klaster): raspoređuje ih, nadzire, zamjenjuje pokvarene i skalira. Cijeli sistem je deklarativan: manifest (YAML) opisuje željeno stanje, a *kontroleri* unutar klastera neprekidno usklađuju stvarno stanje s njim.

OBJEKT	ULOGA
Pod	Najmanja jedinica: jedan ili više kontejnera koji dijele mrežu i volumene. Prolazan je.
Deployment	Održava traženi broj kopija Poda (`replicas`) i vodi izmjene verzija (*rolling update*).
Service	Stabilna adresa i balansiranje prema Podovima odabranim po oznakama (`labels`).
ConfigMap / Secret	Konfiguracija i osjetljive vrijednosti odvojene od slike. Secret je samo *base64* kodiran, **nije** šifrovan: treba mu dodatna zaštita.
Namespace	Logička podjela klastera.

> **POREĐENJE S TERRAFORMOM**
> Terraform usklađuje stanje *na zahtjev* (kad se pokrene `apply`), a drift se otkriva samo tada. Kubernetes ga usklađuje *neprekidno*: ako se Pod ugasi, kontroler ga odmah ponovo napravi. Stanje koje Terraform čuva u `tfstate` datoteci, Kubernetes drži u svojoj bazi (`etcd`). Zajednička ideja: željeno stanje zapisano kao kod, sistem ga provodi.

Primjer 5 (kratki): Deployment, Service i ConfigMap

Lokalni klaster pravi `kind` ("Kubernetes in Docker": čvorovi klastera su Docker kontejneri), pa nije potreban nikakav oblak.

bash + yaml — Isječak 11 · web.yaml

```
mkdir -p ~/iac/s3/p5 && cd ~/iac/s3/p5
cat > web.yaml <<'EOF'
apiVersion: v1
kind: ConfigMap
metadata:
  name: web-index
data:
  index.html: |
    <h1>RuO: nginx na Kubernetesu</h1>
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2
  selector:
    matchLabels: { app: web }
  template:
    metadata:
      labels: { app: web }
```

```
    spec:
      containers:
        - name: nginx
          image: nginx:1.27-alpine
          ports: [{ containerPort: 80 }]
          readinessProbe:
            httpGet: { path: /, port: 80 }
            periodSeconds: 5
          resources:
            requests: { cpu: 50m, memory: 32Mi }
            limits:   { memory: 64Mi }        # postaje cgroup memory.max (sedmica 1)
          volumeMounts:
            - { name: index, mountPath: /usr/share/nginx/html }
      volumes:
        - name: index
          configMap: { name: web-index }
---
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector: { app: web }
  ports: [{ port: 80, targetPort: 80 }]
EOF

kind create cluster --name ruo
kubectl apply -f web.yaml
kubectl get pods -o wide
kubectl port-forward svc/web 8081:80 &
sleep 2; curl -s localhost:8081
```

izlaz (skraćeno)

```
NAME                   READY   STATUS    RESTARTS   AGE
web-7c9d6b5f4-8xk2p    1/1     Running   0          15s
web-7c9d6b5f4-qz4tm    1/1     Running   0          15s
<h1>RuO: nginx na Kubernetesu</h1>
```

bash — Isječak 12 · samoizlječenje i deklarativne izmjene

```
kubectl delete pod $(kubectl get pods -o name | head -1)   # ubij jedan Pod
kubectl get pods                                           # Deployment ga je već zamijenio

sed -i 's/replicas: 2/replicas: 4/' web.yaml
kubectl diff -f web.yaml                                   # šta bi se promijenilo (kao terraform plan)
kubectl apply -f web.yaml                                  # nova željena vrijednost
kubectl get deploy web

kill %1; kubectl delete -f web.yaml; kind delete cluster --name ruo
```

Brisanje Poda ne mijenja željeno stanje (`replicas: 2`), pa kontroler napravi novi: to je samoizlječenje koje Compose i Terraform nemaju bez ponovnog pokretanja. Izmjena broja kopija uradi se u *fajlu*, ne naredbom, a `kubectl diff` i `kubectl apply` imaju istu ulogu kao `plan` i `apply`.

Šta smo dobili i šta dalje

PITANJE	ODGOVOR
Zašto Ansible modul `apt` ne instalira nginx pri drugom pokretanju?	Modul poredi željeno ("prisutan") sa stvarnim stanjem i radi samo razliku. To je idempotentnost.
Čemu služi handler?	Izvršava se samo kad je neki zadatak stvarno promijenio stanje, npr. reload servisa nakon izmjene konfiguracije.
Zašto prefiks `vault_` za tajne varijable?	Šifrovane vrijednosti su odvojene i vidljive u kodu po imenu, a obična varijabla na njih upućuje.
Šta Terraform predaje Ansible-u?	Inventory: spisak mašina i njihovih adresa iz izlaza resursa.
Zašto se u Dockerfileu prvo kopira `requirements.txt`?	Da sloj sa zavisnostima ostane u kešu dok se zavisnosti ne promijene.
Po čemu se Kubernetes razlikuje od Terraforma u usklađivanju stanja?	Kontroleri usklađuju neprekidno, a Terraform samo kad se pokrene `apply`.

Zadaci za samostalni rad

1. U primjeru 1 dodati drugi zadatak koji kreira korisnika `deploy` i pokrenuti playbook dva puta; objasniti zašto je drugi put `changed=0`.
2. U roli iz primjera 2 dodati varijablu `naslov` u `group_vars/web/vars.yml` i potvrditi da nadjačava `defaults`. Pokrenuti `ansible-vault encrypt_string` i ubaciti novu tajnu u `vars.yml`.
3. U primjeru 3 napraviti dvije mašine (`for_each`) tako da inventory ima obje, pa pokrenuti isti playbook.
4. U Compose datoteku dodati treću uslugu (npr. `nginx` kao obrnuti proxy ispred `web`) i ukloniti objavljeni port s `web`.
5. Isti sistem iz primjera 4 napisati kao Kubernetes manifest (Deployment + Service za `web` i `redis`). Sliku učitati u kind pomoću `kind load docker-image ruo/brojac:1.0 --name ruo`.

> **SLJEDEĆA SEDMICA**
> Mašine i usluge su definisane kodom. Sedmica 4 uvodi mrežu i vatrozid kao kod: VPC/subnet, routing i NAT, nftables i security grupe te segmentaciju web/app/db s bastion hostom.